Using python-periphery library on DEBIX

Version: V1.0（2023-09）

Compiled by: Polyhex Technology Company Limited（http://www.polyhex.net/）

DEBIX supports the use of the python-periphery library for direct access to peripherals at the application layer. Executing all the python code in this article requires switching to the root user.

First, install the dependent libraries:

```
sudo apt update
sudo apt install python3-pip
pip install python-periphery
sudo apt -y install gpiod libgpiod-dev
```

GPIO

DEBIX on-board resources are abundant, the recommended pins as GPIOs are listed in the table below:

GPIOx	GPIOCHIPn	OFFSET
GPIO1_IO11	/dev/gpiochip0	11
GPIO1_IO12	/dev/gpiochip0	12
GPIO1_IO13	/dev/gpiochip0	13
GPIO5_IO03	/dev/gpiochip4	3
GPIO5_IO04	/dev/gpiochip4	4
GPIO3_IO21	/dev/gpiochip2	21

For GPIOx_IOy, GPIOCHIPn and OFFSET are calculated as follows:

GPIOCHIPn = x - 1

OFFSET = y

Example:

GPIOCHIPn (GPIO5_IO04)= 5 - 1 = 4 i.e. /dev/gpiochip4

OFFSET (GPIO5_IO04) = 4

Important APIs for GPIOs in the python-periphery library:

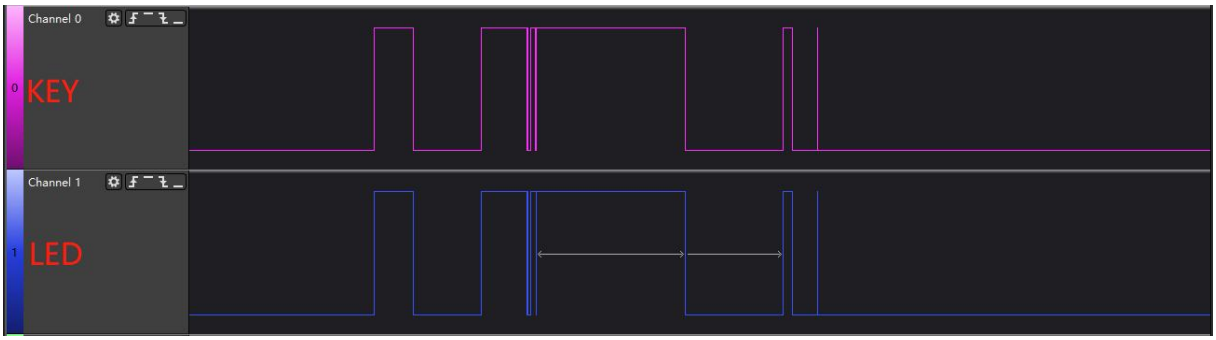
API	Class periphery.GPIO(path,line,direction)	
Formal parameter	**Type**	**Description**
path	str	GPIOCHIP character device path
line	int, str	GPIOLINE number or name
direction	str	GPIO direction
edge	str	GPIO interrupt edge
bias	str	GPIO Line Bias
drive	str	GPIO Line Driver
inverted	bool	GPIO Invert
label	str, None	GPIO Line User Tag
Return Value	GPIO Object	

Example Procedure:

```
from periphery import GPIO
# GPIO1_IO11 as led pin
LED_CHIP = "/dev/gpiochip0"          # CHIP follows the above table
LED_LINE_OFFSET = 11                 # OFFSET follows the above table
# GPIO1_IO12 as key pin
key_CHIP = "/dev/gpiochip0"
key_LINE_OFFSET = 12
led = GPIO(LED_CHIP, LED_LINE_OFFSET, "out")
key = GPIO(key_CHIP, key_LINE_OFFSET, "in")
try:
    while True:
        led.write(key.read())
finally:
    led.write(True)
    led.close()
      key.close()
```

Result output:

Connect the LED (GPIO1_IO11) and KEY (GPIO1_IO12) pins to the logic analyser and change the KEY level, the output is as follows:



It can be seen that there is a normal key jitter, level changes are normal, and real-time is good.

PWM

DEBIX default enable pwm1 as lvds backlight control, default enable pwm2, this section to pwm2 as an example.

1. Enable the pwm2 node in imx8mp-evk.dts.

```
210  &pwm2 {
211      pinctrl-names = "default";
212      pinctrl-0 = <&pinctrl_pwm2>;
213      // status = "disabled";
214      status = "okay";
215  };
```

2. Cancel the pin comment

```
872      pinctrl_pwm2: pwm2grp {
873          fsl,pins = <
874              MX8MP_IOMUXC_GPIO1_IO13__PWM2_OUT  0x116
875          >;
876      };
```

3. Recompile the device tree and replace the original device tree, and restart DEBIX.

```
make dtbs
sudo scp xxx/imx8mp-evk.dtb /boot/
sudo reboot
```

Important API for PWM in python-periphery library:

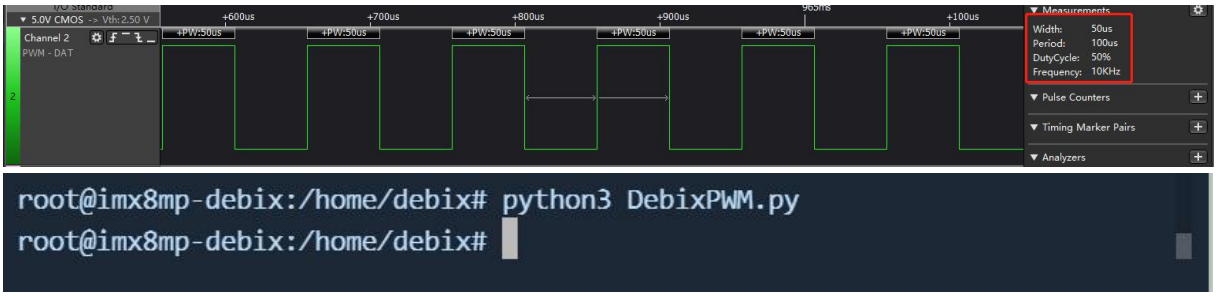
API	class periphery.PWM(chip, channel)	
Formal parameter	**Type**	**Description**
chip	int	PWM chip number
channel	int	PWM channel number
Return Value	PWM Object	

Example Procedure:

```
from periphery import PWM
try:
    # Open PWM2, channel 0, corresponds to PWM2
    pwm2 = PWM(1, 0)
    # Set PWM output frequency to 10 kHz
    pwm2.frequency = 1e4
    # Set the duty cycle to 50%
    pwm2.duty_cycle = 0.5
    # Enable PWM output
    pwm2.enable()
except:
    print("Some errors occur!\n")
finally:
    # Release resources
    pwm2.close()
```

Result output:

Connect the output pin GPIO1_IO13 of pwm2 to the logic analyser, and you can see that the output duty cycle, frequency and other parameters are consistent with the set values:



I2C

DEBIX's I2C1 mounted PMIC, I2C2 mounted camera, I2C3 mounted audio IC, I2C4 mounted EEPROM, HYM8563 and ADS1115. I2C bus can be mounted on more than one device, and and it is recommended to use I2C4 and I2C6.

I2Cx	Device Node
I2C4	/dev/i2c-3
I2C6	/dev/i2c-5

The following is an example of I2C6.

Important API for I2C in python-periphery library:

API	class periphery.I2C(devpath)	
Formal parameter	**Type**	**Description**
devpath	str	The path of device
Return Value	I2C Object	
API	transfer(address, messages)	
Formal parameter	**Type**	**Description**
address	Int	I2c address.
messages	List	I2C.Message list

Example Procedure:

```
from periphery import I2C
I2C4 =   "/dev/i2c-3"
I2C6 =   "/dev/i2c-5"
# Open i2c1 host
i2c = I2C(I2C6)
# Take MPU6050 for example, read data from ID register (0x75), I2C device address is 0x68
msgs = [I2C.Message([0x75]), I2C.Message([0x00], read=True)]
# Enable I2C transfer
i2c.transfer(0x68, msgs)
# Print the received message
print("ID: 0x{:02x}".format(msgs[1].data[0]))
# When the operation is complete, shut down the I2C host to free up resources.
i2c.close()
```

Result output:

The device address of this experiment is 0x68, and the ID of this module is also 0x68. connect the SDA and SCL of the MPU6050 module to I2C6_SDA,I2C6_SCL, and connect the logic analyser:

```
root@imx8mp-debix:/home/debix# sudo python3 DebixI2C.py
ID: 0x68
```

SPI

DEBIX supports user free spidev with spidev1.0 and spidev2.0, which correspond to ECSPI1 and ECSPI2 of the GPIO interface definition in the DEBIX IO Board User Manual respectively.

SPIx	Device Node
ECSPI1	/dev/spidev1.0
ECSPI2	/dev/spidev2.0

The following is an example of ECSPI1.

Important API for SPI in python-periphery library:

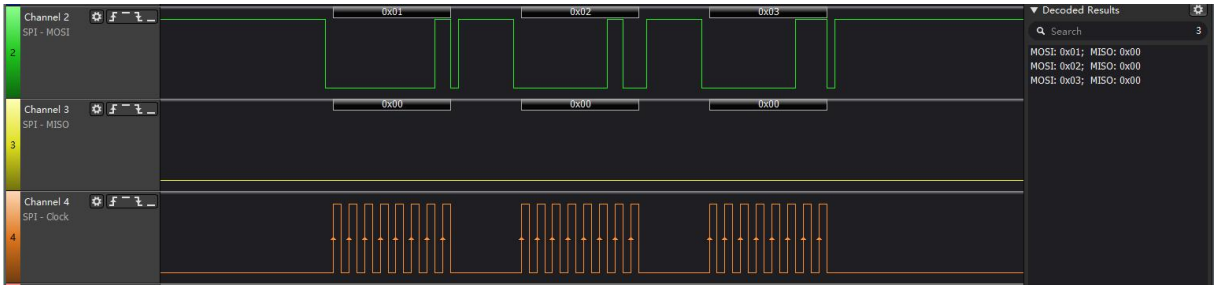
API	Class periphery.SPI(devpath, mode, max_speed, bit_order='msb', bits_per_word=8, extra_flags=0)	
Formal parameter	**Type**	**Description**
devpath	str	The path of device
mode	int	SPI mode
max_speed	int, float	Maximum Speed (Hz)
bit_order	str	Bit Order
bits_per_word	int	Bits per word
extra_flags	int	Additional spidev flags will be bitwise "or" with SPI mode
Return Value	SPI Object	

Example Procedure:

```
import periphery
ECSPI1 = "/dev/spidev1.0"          # device path definition
ECSPI2 = "/dev/spidev2.0"          # device path definition
if __name__ == "__main__":
    # Create SPI object and configure SPI device path, mode and max speed
     spi = periphery.SPI(ECSPI1, mode=0, max_speed=1000000)  # Use mode 0, and
max_speed 1MHz
     # Data to be sent (a list of bytes)
    tx_data = [0x01, 0x02, 0x03]
    try:
        # call spi.transfer method for data transfer
        rx_data = spi.transfer(tx_data)
        # Print the received data
        print("Received data:", rx_data)
    except Exception as e:
        print("Error:", str(e))
    finally:
        # Close the SPI object
        spi.close()
```

Result output:

Connect ECSPI1 to the logic analyser and the output is as follows:



UART

DEBIX uart1 is used for Bluetooth communication, uart2 is used as system DEBUG, and uart3 and uart4 are available for user, which correspond to UART3 and UART4 in the definition of GPIO interface in the DEBIX IO Board User Manual respectively.

UARTx	Device Node
UART3	/dev/ttymxc2
UART4	/dev/ttymxc3

Important API for UART in python-periphery library:

API	class periphery.Serial(devpath, baudrate, databits=8, parity='none', stopbits=1, xonxoff=False, rtscts=False)	
Formal parameter	**Type**	**Description**
devpath	str	Device path
baudrate	int	Baud rate
databits	int	Data bits
parity	str	Parity bit
stopbits	int	Stop bits
xonxoff	bool	Software flow control
rtscts	bool	Hardware flow control
Return Value	Serial Object	

Example Procedure:

```
from periphery import Serial
# UART device path
UART3 = "/dev/ttymxc2"
UART4 = "/dev/ttymxc3"
try:
    # Requested serial resource is /dev/ttymxc2, set serial baud rate to 115200, data bit is 8, no parity bit, stop bit is 1, no flow control is used
    serial = Serial(
        UART3,
        baudrate=115200,
        databits=8,
        parity="none",
        stopbits=1,
        xonxoff=False,
        rtscts=False,
    )
    # Send byte stream data using the requested serial port: "Hello Debix!\n"
     serial.write(b "Hello Debix!\n")
    # Read the data received by the serial port, the function returns the condition that one of the two meets: one, read to 128 bytes; two, reading time more than 1 second
    buf = serial.read(128, 1)
    # Note: Python read data type: bytes
    data_strings = buf.decode("utf-8")
    # Print the amount of data read and the contents of the data
    print("read {:d} byte(s): {:s}".format(len(buf), data_strings))
finally:
    # Release the requested serial port resources
    serial.close()
```

Result output:

Connect UART3_Tx and UART3_Rx for loopback test and connect logic analyser.

```
root@imx8mp-debix:/home/debix# sudo python3 DebixUART.py
read 13 byte(s): Hello Debix!
```